

Proyecto #1: Gestor de Procesos

Estudiante: Diego Araya

Curso: IC-6600 Principios de Sistemas Operativos

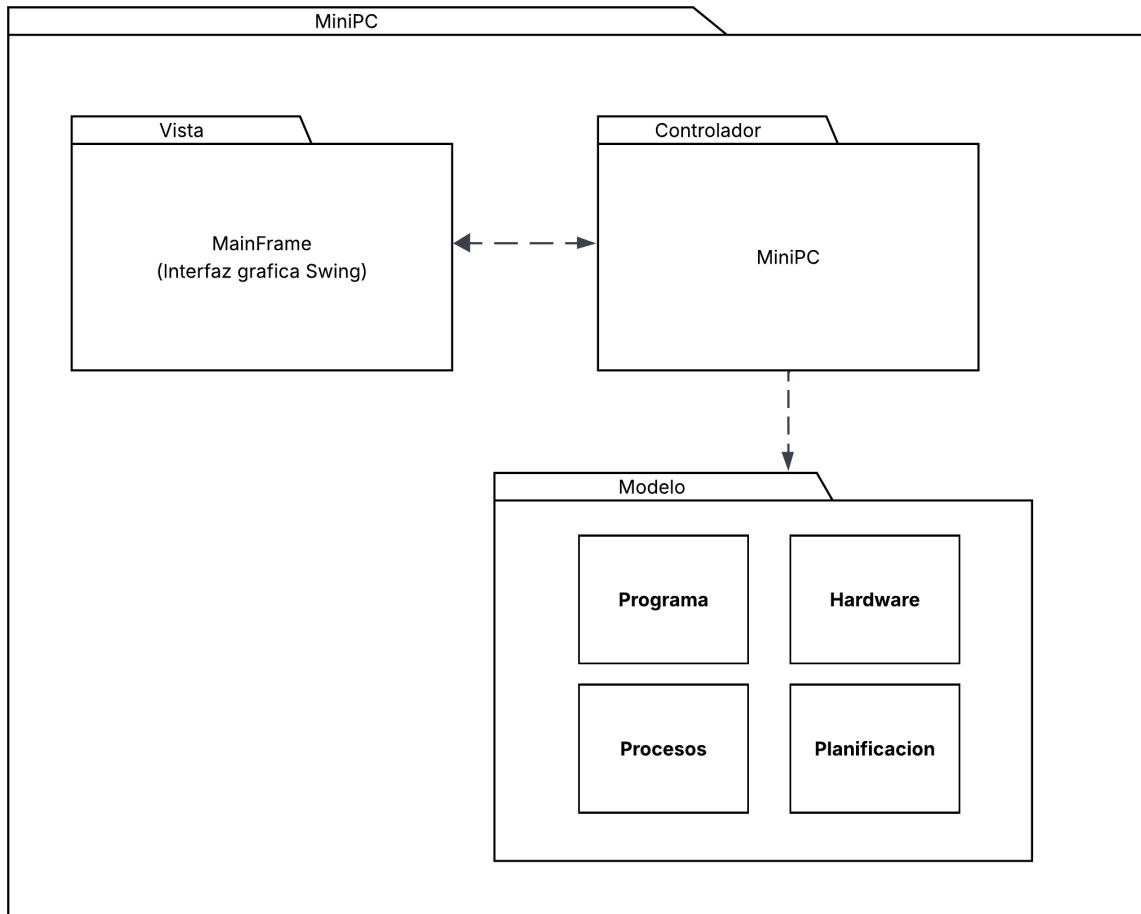
Profesor: Ing. Cristian Campos Agüero

Tecnológico de Costa Rica, Centro Académico Limón

Visión general

En el proyecto se desarrolló el simulador de una minicomputadora en Java con Swing y usando NetBeans. El programa carga uno o varios archivos con instrucciones tipo ensamblador, los convierte en procesos y los ejecuta sobre un CPU simulado. La ventana muestra todo el tiempo la lista de procesos con su estado, la memoria principal, el disco, el BCP del proceso que está usando el CPU, una pantalla para las salidas y un campo de teclado para las entradas. Cada clic en el botón Siguiente representa un segundo de CPU, y el botón Ejecutar repite ese avance hasta que no queda nada por correr.

El código está dividido en tres paquetes, siguiendo el patrón Modelo Vista Controlador. La Vista tiene solo la ventana principal. El Controlador recibe los clics de los botones, le pide trabajo al modelo y actualiza la ventana con el resultado. El Modelo contiene toda la lógica del sistema operativo simulado: el ensamblador, el CPU, la memoria, el disco, el BCP, el planificador y el despachador. La Vista conoce al Controlador solo porque el método main está dentro de la ventana y es el que crea al Controlador al arrancar la aplicación. Los tamaños de memoria y de disco, y los porcentajes de cada zona, no están escritos en el código sino en un archivo config.json que se lee al iniciar.



Funcionamiento del sistema

Todo empieza cuando se cargan los archivos. El Ensamblador los revisa línea por línea y verifica que el operador exista, que los registros sean válidos, que los valores estén dentro del rango de 8 bits con signo, que las comas estén bien puestas y que cada instrucción tenga la cantidad correcta de operandos. Si algo falla, muestra la línea y el motivo, y no se carga ninguno de los archivos seleccionados. Cada línea correcta se convierte en una Instrucción que lleva su peso. Después los programas se guardan en el disco, en una zona de programas que empieza con un índice con el nombre y la dirección de cada archivo, y los trabajos pasan a una cola de trabajos en el orden en que llegaron.

El Planificador implementa FCFS: toma el primer trabajo de la cola y lo admite a memoria principal, siempre que haya espacio y que no haya ya cinco procesos residentes. La memoria principal está dividida en una zona de kernel, que en mi configuración es el 60 por ciento, y una

zona de usuario. En la de usuario se copian las instrucciones de cada programa. En la de kernel se guardan los BCP, y ahí quise que fueran valores legibles y no objetos de Java: cada BCP ocupa 21 posiciones de texto, una por atributo, y cada uno de los cinco elementos de la pila tiene su propia posición. Los BCP forman una lista enlazada dentro del mismo kernel, porque uno de sus campos guarda la dirección de memoria donde empieza el siguiente. La clase TablaProcesos mantiene el inicio de esa lista y se encarga de agregar y quitar procesos, arreglando los enlaces. Un proceso puede estar en siete estados: nuevo, listo, ejecutando, bloqueado, listo suspendido, bloqueado suspendido y terminado.

El CPU tiene sus propios registros AC, AX, BX, CX, DX, PC e IR, y ejecuta el ciclo de buscar, decodificar y ejecutar. Cada clic le resta uno al contador de la instrucción actual y solo cuando llega a cero se aplica su efecto, de modo que una instrucción de peso 3 necesita tres clics. Los saltos se calculan como desplazamientos dentro de la lista de instrucciones del proceso, y CMP deja su resultado en una bandera que después consultan JE y JNE. Cada proceso trabaja con su propia pila de tamaño 5.

El planificador no es apropiativo: un proceso corre hasta que termina o hasta que se bloquea. Cuando eso pasa, el Despachador hace el cambio de contexto, copiando los registros del CPU al BCP del proceso que sale y cargando en el CPU los del que entra. El único caso de bloqueo es INT 09H: el proceso queda esperando que el usuario escriba un número entre 0 y 255 en el campo de teclado, y mientras tanto el CPU atiende a otro proceso. Cuando llega el valor, se guarda en DX y el proceso vuelve a estar listo. Si varios procesos esperan teclado, se atienden en el orden en que se bloquearon. INT 10H imprime el valor de DX en la pantalla e INT 20H finaliza el programa.

La memoria virtual vive en el disco, que tiene dos zonas: la de programas, que empieza con el índice de archivos, y la de memoria virtual. Cuando entra un trabajo y ya hay cinco procesos en memoria, o no hay espacio contiguo suficiente, el planificador manda a disco a un proceso que esté listo, el que llegó de último para no romper el orden FCFS. Su programa se copia a la zona de memoria virtual, se libera su espacio en memoria principal y su BCP queda en el kernel con el estado listo suspendido, con la base, el límite y el PC apuntando ahora al disco. Cuando otro proceso termina y se libera espacio, el proceso suspendido regresa a memoria principal. Al terminar cada proceso se registra su hora de inicio, su hora final y su duración en segundos, y esa información se muestra en las estadísticas.

Sobre el tema de seguridad, me encargue de modular lo más posible las clases y validar todo lo que entra. La memoria separa el kernel de la zona de usuario y rechaza escrituras fuera de la zona que corresponde. Las instrucciones no pueden leer ni escribir memoria directamente, y para usar la pantalla o el teclado un programa solo puede pasar por las interrupciones. Cada proceso tiene sus propios registros y su propia pila dentro de su BCP, y el CPU solo ejecuta el programa del proceso que tiene montado. Además, se validan los archivos, el archivo de configuración y el valor del teclado, se limpian las posiciones que se liberan, y los recursos tienen límites: cinco procesos en memoria, un máximo de BCP en el kernel y un tope de 10000 segundos cuando se usa Ejecutar.

Analisis de resultados.

Requisito	Estado	Detalle
Interfaz gráfica	Cumplido	Ventana con las tablas de procesos, memoria y disco, el panel del BCP en ejecución, la pantalla, el campo de teclado y la configuración.
Ejecución en dos modos	Cumplido	Siguiente avanza un segundo de CPU y Ejecutar corre todos los procesos hasta que terminen.
Validación de archivos	Cumplido	Acepta uno o varios archivos, valida sintaxis, registros, rangos y comas, e informa la línea y el motivo. Si uno falla, no se carga ninguno.
Teclado y pantalla	Cumplido	Entrada numérica de 0 a 255 validada, y salida de INT 10H en la pantalla simulada.
Memoria principal	Cumplido	Zona de kernel y zona de usuario con tamaño y porcentaje configurables, y los BCP guardados como texto legible en el kernel.
CPU	Cumplido	Registros propios, ciclo de buscar, decodificar y ejecutar, y manejo del peso de cada instrucción.

Requisito	Estado	Detalle
Almacenamiento secundario	Cumplido	Disco configurable con zona de programas, índice de archivos y zona de memoria virtual. El índice tiene diez entradas, así que en una sesión se pueden cargar diez archivos antes de limpiar.
Lista de trabajos y planificador	Cumplido	Cola de trabajos, planificador FCFS no apropiativo y estructura de lista de procesos como lista enlazada de BCP dentro del kernel.
Despachador y cambio de contexto	Cumplido	Guarda y restaura los registros entre el CPU y el BCP cada vez que un proceso termina o se bloquea.
Memoria virtual	Cumplido	Máximo de cinco procesos en memoria. Los demás se mandan a disco y regresan cuando se libera espacio.
Estadísticas	Cumplido	Al terminar cada proceso se muestra su hora de inicio, su hora final y su duración en segundos.
Configuración externa	Cumplido	Tamaños de memoria y disco, y porcentajes de cada zona, en config.json. Desde la ventana se pueden cambiar, pero solo durante esa sesión.
Ejecución de instrucciones	Parcial	Funcionan 20 de las 21 instrucciones de la tabla, con sus pesos. Falta INT 21H.
Interrupciones	Parcial	INT 10H, INT 09H e INT 20H funcionan, con bloqueo del proceso en INT 09H. INT 21H no realiza ninguna operación.
BCP	Parcial	Guarda estado, registros, PC, IR, pila, base, límite, prioridad, tiempo de inicio, tiempo de CPU y enlace al siguiente BCP. El CPU donde se ejecuta solo se muestra en el panel, el campo de archivos abiertos siempre está vacío y la prioridad no se usa.

Requisito	Estado	Detalle
Estados del proceso	Parcial	Están definidos los siete, pero nuevo y bloqueado suspendido casi no se alcanzan: un proceso pasa a listo apenas se crea, y solo se mandan a disco procesos que están listos.
Pila con control de desbordamiento	Parcial	Cada proceso tiene su pila de tamaño 5, con cada elemento en su propia posición del BCP. El desbordamiento se detecta, pero no se atiende; el proceso queda trabado en esa instrucción. Lo mismo pasa con un POP sobre una pila vacía.
Protección y seguridad	Parcial	Hay aislamiento entre kernel y usuario y entre procesos, acceso a dispositivos solo por interrupciones y validación de todas las entradas. No se implementó revisar la base y el límite durante la ejecución, controlar los saltos fuera del programa y el desbordamiento de las operaciones aritméticas.
INT 21H, manejo de archivos	No cumplido	El programa acepta la instrucción y gasta sus cinco segundos de CPU, pero no realiza ninguna operación de archivos.